Optimization Grand Challenge 2026 (OGC 2026)
The Grand Shipyard Puzzle: *Pack the Block, Beat the Clock*

OGC 2026 Organizing Committee

May 31, 2026 (`Document Version 1.1`)

Abstract

The Optimization Grand Challenge (OGC) 2026 addresses a spatial block scheduling problem drawn from shipbuilding. In a shipyard, each structural block of a ship is built inside a fixed workspace called a bay. Blocks are three-dimensional objects with irregular, layered shapes, so placing them in a bay without overlap is itself a hard geometric problem. On top of this, every block carries a release date, a processing time, and a due date, and bay space is shared—so a block may have to wait, or stay longer than needed, because of other blocks around it. An algorithm must therefore decide, for each block, which bay it goes to, where and how it sits, and when it enters and leaves—all at once. This document gives a formal problem definition, a participation guide, and the evaluation criteria.

Document updates. This document may be revised to correct errors or clarify ambiguities. The latest version is always available on the competition website, and the version number on the title page indicates which revision you are reading. Participants are responsible for checking the website for updates; significant changes will also be announced there.

1 Problem Description

1.1 Terminology

- **Bay**: A physically partitioned workspace in a shipyard where operations are performed. Each bay $j \in M$ has a fixed size $W_j \times H_j$, where $M := \{1, 2, \ldots, m\}$ is the set of bays.
- **Block**: A production unit representing a large structural component of a ship. Each block $i \in N$ consists of K_i polygonal layers forming a 3D structure, where $N := \{1, 2, \ldots, n\}$ is the set of blocks. A block can be rotated, and the number of possible orientation options is given as O_i.
- **Release date** (R_i): The earliest time at which block i can start processing.
- **Due date** (D_i): The desired completion time of block i.
- **Processing time** (P_i): The time required to process block i.
- **Entry time** ($ENTRY_i$): The time at which block i is placed into a bay.
- **Exit time** ($EXIT_i$): The time at which block i leaves the bay.
- **Tardiness** (T_i): The delay beyond the due date, defined as

$$T_i = \max(0, EXIT_i - D_i).$$

- **Workload** (L_i): The total workload required during the processing of block i.
- **Preference** (S_{ij}): The preference score of block i to be assigned to bay j.

1.2 Problem Setting

In a shipyard, blocks undergo several production stages, some of which are carried out within bays. Each bay is a fixed workspace where resources such as cranes, labor, and transportation paths are shared. Blocks must be assigned to bays and processed while satisfying both temporal and spatial constraints. Each bay can accommodate multiple blocks simultaneously, provided that they do not overlap in space. Each block has its own release date, processing time, and due date. Therefore, the assignment and scheduling decisions must consider both spatial and temporal feasibility simultaneously.

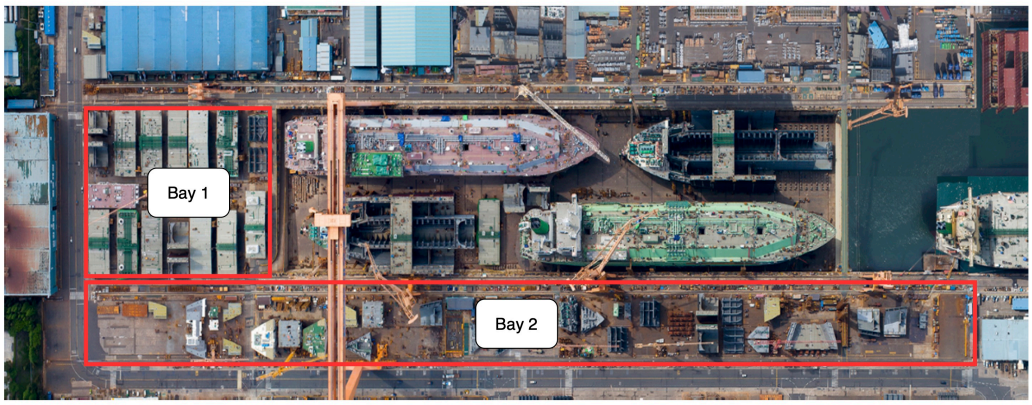

Figure 1: Real shipyard layout with multiple bays and block operations

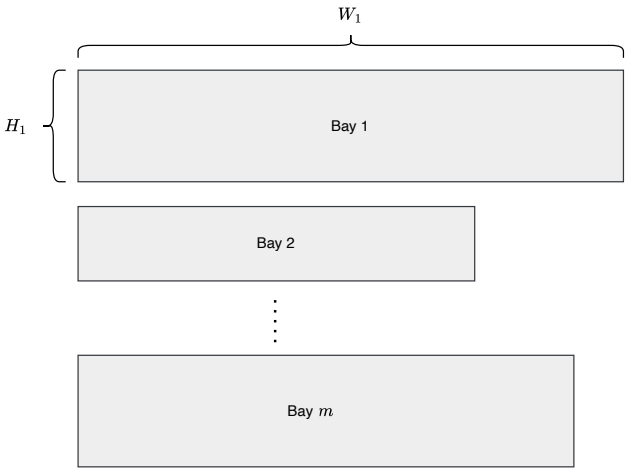

Figure 2: Bays

1.3 Bays

Each bay is an independent workspace, and there exists physical or operational separation between bays. A typical shipyard has multiple bays as shown in Figure 1. We assume that the shipyard consists of m bays, each bay j has a size $W_j \times H_j$. For any given block, the following decisions must be made simultaneously:

1. which bay to assign,
2. the position and orientation within the bay,
3. when to put the block on the bay
4. when to pull the block from the bay

We assume that each bay has an independent crane, so any *operations* in the bay can be done separately. There are two operations regarding this year's competition: ENTRY and EXIT, which represent putting a block into the bay and removing it from the bay, respectively. For a given day, the crane operator of each bay would receive a list of operations to conduct on that day. We do not limit the number of operations that can be conducted on a single day, so any number of operations can be performed *before* the day's work. For example, consider a block with a processing time of 3, scheduled to be placed in a bay on day 5 (ENTRY operation on day 5). Then, the earliest day the block is ready for the EXIT operation is day 8. In other words, the block can be processed from day 5 to day 7. Note that, in this case, the order of operations is also important because each operation would make the bay look different. Nonetheless, we assume that all EXIT operations must be done before any ENTRY operations. Among the same type of operations, the algorithm should determine the order in which to conduct them.

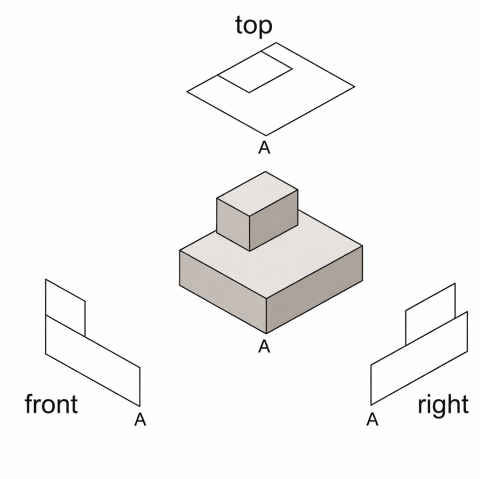

Figure 3: A block with two layers

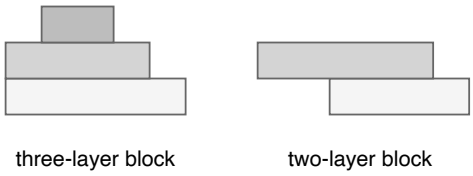

Figure 4: Two blocks with different numbers of layers (front view)

1.4 Blocks

We are given a total of n blocks. Each block is assumed to consist of multiple polygonal layers forming a three-dimensional structure. The polygon defining each layer is not necessarily convex. For example, a block composed of two layers, where each layer has a rectangular shape, is illustrated in Figure 3. The top view shows the planar shape as seen from above, the front view shows the shape as seen from the front, and the right view shows the shape as seen from the right side.

Each block i is assumed to consist of K_i layers, and all layers are assumed to have the same height. For example, the front views of a three-layer block and a two-layer block are shown in Figure 4. Note that there cannot be an "empty" layer. Moreover, any two adjacent layers are guaranteed to be physically attached, so "floating" layers are impossible.

When a block is placed, the *orientation* of the block should also be determined. For this competition, we provide problem instances with predetermined numbers of possible orientations for each block, denoted by O_i for block i. For example, Figure 5 shows a block with eight possible orientation options.

Each block i also has a workload L_i and preference scores for bays S_{ij}. The workload indicates how much work should be done for the block, and a solution with less variance in total workloads across the bays is preferred. The preference score S_{ij} indicates how much block i is preferred for assignment to bay j.

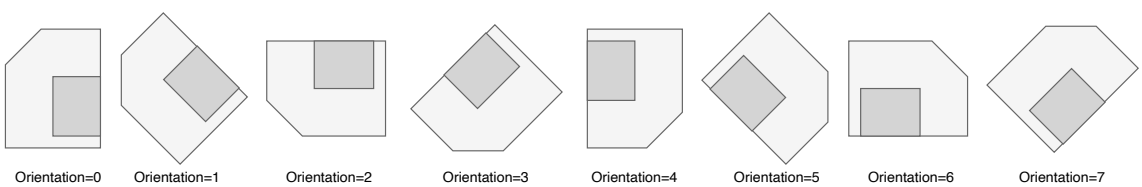

Figure 5: Possible orientation options of a block

1.5 Decisions to Make

For each block i, the following decisions must be made:

- Assignment to a bay j
- Placement position within the bay
- Orientation selection
- Entry time $ENTRY_i$ and exit time $EXIT_i$

1.6 Constraints

There are three main groups of constraints. The first constraint concerns the assignment of every block to a bay, with ENTRY/EXIT operations assured. The second is the temporal constraint, which ensures that work on the block is performed only after the block has been available for at least the required processing time. The third group of constraints concerns the spatial feasibility of the blocks within the bay. Because the crane operations can be performed before the day's working time, a block i with $ENTRY_i$ and $EXIT_i$ occupies the assigned bay during $[ENTRY_i, EXIT_i)$.

1.6.1 Assignment Constraints

Assignment constraint For every block, it must be assigned to a single bay. There cannot be any unassigned blocks.

Operation constraint For every block, it must have exactly one ENTRY operation and one EXIT operation. A block cannot be moved or rotated once placed in the bay.

1.6.2 Temporal Constraints

Release date constraint A block cannot start before its release date:

$$ENTRY_i \geq R_i.$$

Processing time constraint A block's work must be completed before exit:

$$EXIT_i - ENTRY_i \geq P_i.$$

Note that the block may "remain" in the bay after the required processing time has elapsed. This can happen if other blocks prevent it from exiting. If the exit time $EXIT_i$ exceeds the due date D_i, a positive tardiness occurs. Typically, a block has $D_i - R_i > P_i$, which implies there is some "slack" time to determine better ENTRY and EXIT times for the block, considering the other blocks in the bay.

1.6.3 Spatial Constraints

Let $P_{i,l}^{o,x,y}$ denote the polygon of layer l of block i positioned at location (x, y) with orientation o. Consider two blocks i_1 and i_2 located in the same bay on the same day. We say there is a collision between two blocks i_1 and i_2 if $\text{int}(P_{i_1,l}^{o_1,x_1,y_1}) \cap \text{int}(P_{i_2,l}^{o_2,x_2,y_2}) \neq \emptyset$, where $\text{int}(\cdot)$ represents the set of interior points of the (closed) polygon. In other words, two blocks are collision-free if $\text{int}(P_{i_1,l}^{o_1,x_1,y_1}) \cap \text{int}(P_{i_2,l}^{o_2,x_2,y_2}) = \emptyset$ for any layer l. Note that we allow two blocks to share polygon edges to be collision-free. We define a function $C(i_1, l_1, o_1, x_1, y_1, i_2, l_2, o_2, x_2, y_2)$ as

$$C(i_1, l_1, o_1, x_1, y_1, i_2, l_2, o_2, x_2, y_2) := \begin{cases} 0, & \text{if } \text{int}(P_{i_1,l_1}^{o_1,x_1,y_1}) \cap \text{int}(P_{i_2,l_2}^{o_2,x_2,y_2}) = \emptyset, \\ 1, & \text{otherwise.} \end{cases}$$

For any date t and bay $j \in M$, we define the set

$$N(t, j) := \{i \in N : ENTRY_i \leq t < EXIT_i \text{ and } i \text{ is assigned to } j\}.$$

That is, the set $N(t, j)$ is a subset of blocks that are assigned to the same bay j and stay in the bay at the time t.

Bay containment constraint Each block must be fully contained within the assigned bay. Figure 7(b) shows two blocks violating this constraint.

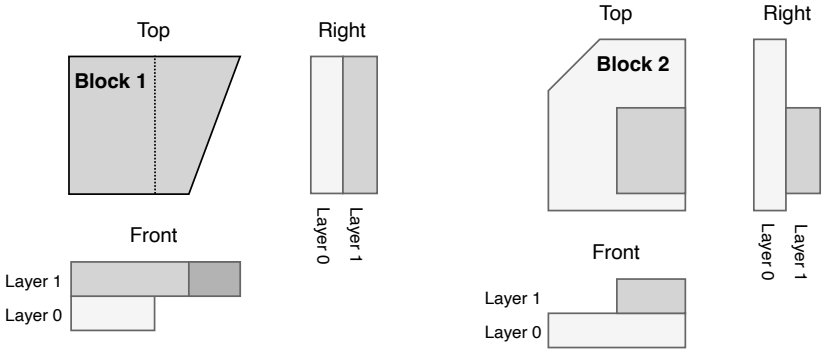

Figure 6: Two blocks with two layers orientation 0

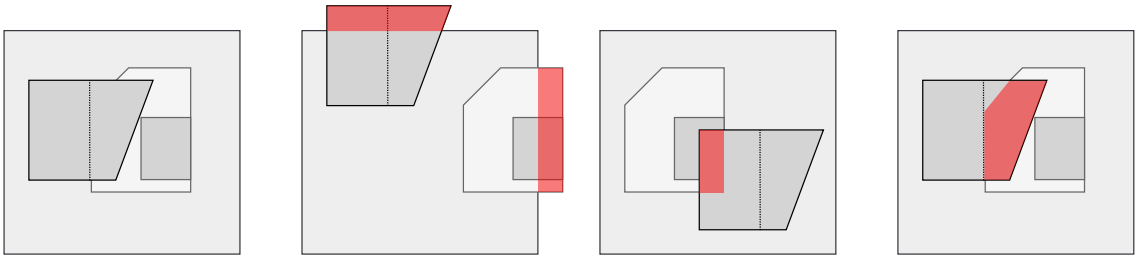

Figure 7: Examples of spatial constraints

Layer collision-free constraint For any date t, bay j, and any two distinct blocks $i_1, i_2 \in N(t,j)$, the decision variables $(o_1, x_1, y_1, o_2, x_2, y_2)$ must satisfy $C(i_1, l, o_1, x_1, y_1, i_2, l, o_2, x_2, y_2) = 0$ for all $l = 1, 2, \ldots, \min\{K_{i_1}, K_{i_2}\}$. Plainly speaking, two blocks in the same bay on the same date are spatially feasible if their layers at the same level are collision-free. For example, Figure 7(a) shows that the two blocks are spatially feasible even though they appear to overlap from the top view. On the other hand, Figure 7(c) shows that two blocks are spatially infeasible.

Crane operation constraint To place or remove the blocks in the bay, the crane should be able to move the blocks vertically without any interference. Assume that, for any date t and bay j, either crane operation (ENTRY or EXIT) of block i_1 is feasible only if $C(i_1, l_1, o_1, x_1, y_1, i_2, l_2, o_2, x_2, y_2) = 0$ for any $l_1 \leq l_2$, where $i_2 \in N(t,j)\backslash\{i_1\}$, $l_1 = 1, 2, \ldots, K_{i_1}$, and $l_2 = 1, 2, \ldots, K_{i_2}$. Plainly speaking, to put or pull a block, the block's layer must be spatially feasible with the layers of other blocks at the same or higher levels. For example, Figure 7(d) shows that Block 2 violates the crane operation constraint if it is put or pulled when Block 1 already exists. On the other hand, Block 1 can be craned even if Block 2 exists.

1.7 Objective Function

The objective is to minimize a weighted combination of three criteria:

Total tardiness The sum of the total tardiness of blocks.

$$Z_1 := \sum_{i=1}^{n} T_i,$$

Workload imbalance Maximum normalized workload imbalance across bays.

$$Z_2 := \left\lfloor \max_{j_1, j_2 \in M : j_1 \neq j_2} \left| u_{j_1} \sum_{i \in N(j_1)} L_i - u_{j_2} \sum_{i \in N(j_2)} L_i \right| \right\rfloor,$$

where $N(j) \subseteq N$ is the set of blocks assigned to bay j and u_j is the weight assigned to the bay where each block is placed. This value is defined as the average area of all bays divided by the area of the specific bay;

$$u_j = \frac{\frac{\sum_{k \in M} W_k \times H_k}{m}}{W_j \times H_j}.$$

A larger bay has a smaller weight value, this reflects that placing blocks in a larger bay reduces congestion, even if the blocks have the same workload.

Total preference score Total sum of preference scores for the assigned bays.

$$Z_3 := \sum_{j \in M} \sum_{i \in N(j)} (S_i^{max} - S_{ij}),$$

where $S_i^{max} := \max_{j \in M}\{S_{ij}\}$. Note that $Z_3 = 0$ if all blocks are assigned to the bay with the most preferred score.

So, we want to minimize the function

$$w_1 Z_1 + w_2 Z_2 + w_3 Z_3,$$

where w_1, w_2, and w_3 are the given nonnegative weight parameters.

2 Problem Instance Definition

The participants are provided with "training" problem instances generated by the OGC committee. These instances are designed to closely mimic real-world shipbuilding problems and include various challenging aspects. For example, some instances have many blocks, while others may have tighter block-release/due-date schedules. We expect the participants to develop their algorithms by considering these various challenges so that the resulting algorithms perform well across a wide range of situations.

A single problem instance is given in `json` format as follows:

```
{
  "name": "training_problem_01",
  "bays": [
    {
      "width": 125,
      "height": 15
    },
    {
      "width": 71,
      "height": 18
    },
    {
      "width": 140,
      "height": 25
    }
  ],
  "blocks": [
    {
      "release_time": 37,
      "due_date": 44,
      "processing_time": 7,
      "workload": 20,
      "bay_preferences": [
        19,
        26,
        55
      ],
      "shape": [
        {
          "orientation": 0,
          "layers": [
            [
              [ 0.0, 0.0 ], [ 0.8093, 5.4723 ], [ 1.2955, 11.0993 ], ...
            ],
            [
              [ -3.0413, 0.3435 ], [ 0.5671, -0.064 ], [ 4.8185, 3.4301 ], ...
            ]
          ]
```

```
        },
        {
          "orientation": 1,
          "layers": [
            [
              [ 0.0, 0.0 ], [ -3.2972, 4.4418 ], [ -6.9323, 8.7645 ], ...
            ],
            [
              [ -2.3934, -1.9076 ], [ 0.4463, 0.3558 ], [ 0.9818, 5.8327 ], ...
            ]
          ]
        },
        ...
      ]
    },
    {
      "release_time": 42,
      "due_date": 49,
      "processing_time": 5,
      "workload": 11,
      "bay_preferences": [
        0,
        72,
        28
      ],
      "shape": ...

    ...
    },
    ...
  ],
  "weights": {
    "w1": 26667,
    "w2": 10,
    "w3": 300
  }
```

Listing 1: Example problem instance

Listing 1 shows an example of a problem instance in the `json` file. The problem instance file consists of four parts: (1) problem name (`"name"`), (2) a list of bays (`"bays"`), (3) a list of blocks (`"blocks"`), and (4) the objective function weights (`"weights"`).

We assume that the bays are indexed from 0 to $m-1$, where m is the number of bays. Each bay $0 \leq j \leq m-1$ may have a different size as shown in the example. For example, the bay 0 has the size of 125×15.

Similarly, the blocks are indexed from 0 to $n-1$. For example, block 0's release date is 37, its due date is 44, and its bay preferences are 19, 26, and 55, i.e., $S_{00} = 19$, $S_{01} = 26$, and $S_{02} = 55$. Note that the preference scores for a single block always sum to 100 across all bays. We also note that all time-related values and preference scores are always integers. The `"shape"` of a block describes the layer polygons for each orientation. Each orientation is a dictionary with an orientation ID (`"orientation"`) and a list of layer polygons (`"layers"`). The orientation ID is always an integer. The list of layer polygons contains the vertex pairs for each layer, ordered from the lowest to the highest. For example, block 0's orientation ID 0 consists of two layers: layer 0 is defined as `[[ 0.0, 0.0 ], [ 0.8093, 5.4723 ], [ 1.2955, 11.0993 ], ...]` and layer 1 is defined as `[[ -3.0413, 0.3435 ], [ 0.5671, -0.064 ], [ 4.8185, 3.4301 ], ...]`. Note that the first vertex of the first layer (layer 0) is always `[0.0, 0.0]`. We call this vertex a *reference* point. All other vertices of the same orientation shape are relative to the reference point. When the algorithm determines the location of a block as (x, y), the locations of the vertices of the block shape inside the assigned bay are translated from the bay's origin by (x, y). Figure 8 illustrates the geometry of the bay and blocks. The block's location is specified by translating the reference point to (x, y) while keeping all other relative positions of the layer's vertices to the reference point unchanged. Therefore, the relative location of a block's reference point is not always at the bottom left when the block's orientation is different, as shown in Figure 8.

We note that a layer's vertex, other than the reference point, can have a fractional location, with up to four significant digits. Because layer shapes can be quite irregular, numerical instability may arise when checking for spatial collisions between blocks. We strongly recommend using the functions in the provided `utils.py` at least at the final stage of your algorithm, as the evaluation server will also use them to check the feasibility of the resulting solutions.

All indices, including bays and blocks, are 0-based in the problem instance and the algorithm

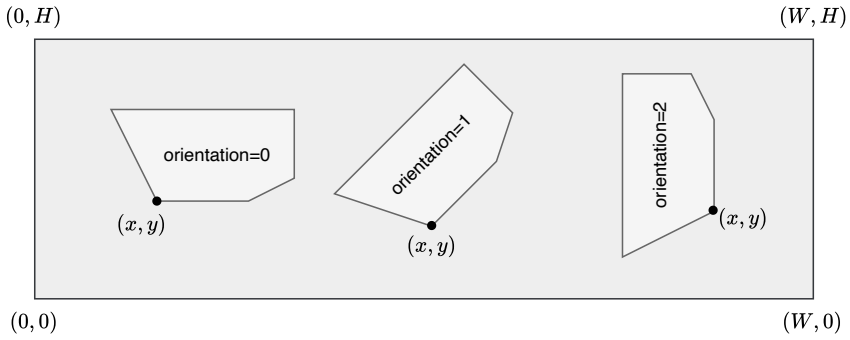

Figure 8: Geometry of bay and blocks

solution. Please note that, in contrast, the formal problem definition in Section 1 uses 1-based indices. We made this distinction to keep the algorithm code and utility functions more Pythonic.

3 Algorithm Submission and Evaluation

The teams should submit their algorithm code to the evaluation server by emailing it to submission@optichallenge.com. Algorithms are evaluated on the "hidden" problem instances on the dedicated server, and the results are ranked against other teams' results to build leaderboards, which will be available on the competition webpage.

3.1 Algorithm Submission

To submit algorithm code to the evaluation server, the following requirements must be met:

- The email should be sent to submission@optichallenge.com.
- The email should be sent from the email address used for registration.
- The email should include a single zip file as an attachment.
- The zip file should include `myalgorithm.py`.
- If `utils.py` file is included in the zip file, it cannot be modified. (It will be overwritten before running the algorithm.)
- The file `myalgorithm.py` should be at the root of the decompressed files (not inside a subfolder).
- The zip file size cannot exceed 15MB.
- A new submission cannot be accepted if the previous accepted submission time is not before the predefined cooldown period (12 hours).

Any submission that does not satisfy any of the above rules will be rejected. The server will perform additional sanity checks (e.g., checking for a broken zip file) before accepting the submission to ensure the submitted algorithm code is valid. Once a submission is accepted, the algorithm code will be executed to solve the hidden instances. All evaluation of results is based on the *latest accepted submission*. However, we note that acceptance of the submission does not guarantee that the algorithm will produce correct results. It may still crash or even fail to execute due to a missing dependency. Regardless of their success or failure, the *latest accepted submission* will be used for the leaderboard, so teams should ensure all submission requirements are met. The competition system will reply to submission emails with the acceptance/rejection status, with a brief description of the reason for rejection if rejected. Note that the teams can resubmit immediately after correcting the cause of the rejection. The cooldown timer starts only after the submission is accepted.

We note that some email service providers block certain potentially malicious file extensions in attachments, such as `*.dll`, `*.vb`, and `*.exe` [1]. It is the team's responsibility to ensure the attachment files can be safely accessed from the evaluation system.

The exact start and end of the submission period will be announced on the competition webpage. There is no limit on the number of submissions allowed during the competition. Nonetheless, the competition committee reserves the right to suspend or even ban teams from submissions if it is determined that the submission system is abused by those teams. We expect the teams to behave fairly and respectfully in using the submission system.

[1] For example, refer to this for Gmail.

3.2 Algorithm Evaluation Server

Teams are expected to develop their algorithms using training problems. The submitted algorithm code is executed on hidden problems for evaluation. The evaluation server specification is:

- AMD Ryzen Threadripper PRO 9955WX
- Ubuntu 24.04 LTS

Algorithms are subject to a time limit, which can vary across problems. The time limits for hidden evaluation problems are disclosed only after the competition ends. We expect the typical time limits to range from a few minutes to half an hour, measured in wall-clock time. Because the participating teams' development environment specs differ from the server specs, algorithms should carefully monitor their internal elapsed time and avoid exceeding limits. Any execution failing to produce the solution within the time limit is treated the same as an algorithm crash or malfunction.

For the sake of fair competition, submitted algorithms are also subject to the following constraints:

- No external internet access during execution
- No access to parent directories above the execution folder
- At most 4 CPU cores (400% CPU usage)
- At most 16GB of memory

Violating these constraints may lead to disqualification. Internet blocking and CPU core limits are enforced using `firejail` and `cpulimit` at evaluation time. Your algorithm must not conflict with these tools.

Submitted algorithms are run on hidden problems, and results are aggregated hourly. At each aggregation point, a leaderboard is generated from algorithms that produced results on all problems and is then published on the website. We note that the "submission time" may differ from the "algorithm execution complete time" because the evaluation server maintains a queue of submissions, which are processed on a first-come, first-served basis. The teams will receive reply emails from the submission system once the algorithm has finished executing.

3.3 Leaderboard

The leaderboard ranking is score-based. For each hidden test instance, the following score is assigned based on the algorithm's produced solution.

- -1, if the solution is infeasible, the algorithm is time-limited, or crashes
- $R - nb$, if the solution is feasible, where R is the number of evaluated teams and nb is the number of teams that found a strictly better objective for this problem.

Leaderboard rankings are determined by total points across all problems at the time the leaderboard is built. The feasibility of solutions will be checked using the `check_feasibility()` function in the provided `utils.py` module. Once a solution is proven feasible, the system will also calculate its objective value.

Please be aware that the final winners are determined by considering written reports and presentations in addition to the leaderboard ranking. Therefore, the final judgment is based on three main criteria: leaderboard ranking, technical report, and presentation. The organizing committee reserves the right to adjust the weights for the final criteria. Nonetheless, we strive to balance raw algorithmic performance with novel scientific contributions. Accordingly, novel ideas or theoretical innovations would be appreciated, even if they do not fully translate into algorithmic performance.

4 Algorithm Development and Baseline

This section explains how to write algorithm code for the competition using the provided baseline algorithm as an example.

4.1 Development Environment

Before submission, each team should implement and test the algorithm on their own machines. This section explains how to build a Python environment similar to the evaluation server.

The submitted algorithm will be run in a `conda` environment created with miniforge. The environment file used to create the server conda environment can be downloaded from the competition webpage.

Some notable Python packages and their versions available in the conda environment are:

```yaml
  - python 3.12
  - jupyterlab 4.4.1
  - notebook 7.4.1
  - ipympl 0.9.7
  - pandas 2.2.3
  - networkx 3.4.2
  - scipy 1.15.2
  - scikit-learn 1.6.1
  - numba 0.61.0
  - cython 3.0.10
  - openjdk 17
  - dotnet-runtime 8
  - pyqt6 6.11.0
  - ortools 9.15.6755
  - gurobipy 13.0.2
  - xpress 9.8.1
  - gymnasium 1.2.3
  - torch 2.11.0
  - torchvision 0.26.0
  - tensorflow 2.21.0
  - keras 3.14.0
  - shapely 2.1.2
```

Listing 2: `OGC2026` `conda` environment file

Teams can use other Python packages or libraries by zipping all required files into a single submission archive. Depending on the hardware/OS, environment creation may fail (especially with `pytorch`, `tensorflow`, etc.). If you do not use those problematic packages, remove them from the provided yml file and retry. This conda environment is provided to increase compatibility with the evaluation server. It is not mandatory, and exact behavior may still differ by OS. General conda/Python issues should be handled by teams.

You can request the installation of additional packages required for your algorithm. The organizing committee will decide whether to install the requested packages. The competition will establish a Discord server for participant questions and requests. Please refer to the competition webpage for instructions on joining the Discord server.

Note that although machine-learning-related packages are readily available, there is no dedicated GPU on the evaluation server. We expect ML-related algorithms to be permitted as long as they run on a CPU.

With generous support from Gurobi and FICO, submissions can use the following commercial MIP solvers:

- Gurobi 13.0.2
- Xpress 9.8.1

4.2 Baseline Algorithm

We provide a simple greedy algorithm, available on the competition webpage, to help teams in developing their own algorithms. The `myalgorithm.py` is the entry point of your algorithm. The evaluation server will seek this file and function `algorithm(prob_info, timelimit=60)` as shown in Listing 3.

```python
def algorithm(prob_info, timelimit=60):
    """
    This is a template for the custom algorithm.
    The function signature must not be changed or removed, but you can define extra functions or modules that are used in this function.
    The `prob_info` is a dictionary containing the problem information, and `timelimit` is the algorithm's time limit in seconds.
    The function should return a solution in the format specified in the problem statement.
    Please refer to baseline_greedy.py for an example implementation of a simple greedy algorithm. You can use it as a starting point or reference for your own algorithm.
    """
    # You can import other modules or define extra functions here.
    import baseline_greedy

    return baseline_greedy.greedyalgorithm(prob_info, timelimit)
```

Listing 3: Example `myalgorithm.py`

The function `algorithm(prob_info, timelimit=60)` should return a Python dictionary containing solution information for the given problem `prob_info`.

The solution dictionary should look like:

```
{
  "operations": {
    "0": [
      {
        "type": "ENTRY",
          "block_id": 48,
          "bay_id": 1,
          "x": 0,
          "y": 0,
          "orient_idx": 0
      },
        {
          "type": "ENTRY",
          "block_id": 53,
          "bay_id": 0,
          "x": 0,
          "y": 0,
          "orient_idx": 0
      }
    ],
    "1": [
        {
          "type": "ENTRY",
          "block_id": 19,
          "bay_id": 2,
          "x": 37,
          "y": 0,
          "orient_idx": 2
        },
        {
          "type": "ENTRY",
          "block_id": 66,
          "bay_id": 2,
          "x": 22,
          "y": 0,
          "orient_idx": 0
        },
        ...
      ],
      ...
      "28": [
        {
          "type": "EXIT",
          "block_id": 25,
          "bay_id": 2
        },
        {
          "type": "EXIT",
          "block_id": 31,
          "bay_id": 1
        },
        {
          "type": "ENTRY",
          "block_id": 15,
          "bay_id": 0,
          "x": 106,
          "y": 0,
          "orient_idx": 0
        },
        {
          "type": "ENTRY",
          "block_id": 35,
          "bay_id": 2,
          "x": 13,
          "y": 0,
          "orient_idx": 0
        },
        {
          "type": "ENTRY",
```

```
            "block_id": 44,
            "bay_id": 0,
            "x": 19,
            "y": 0,
            "orient_idx": 0
          }
        ],
        ...
    }
}
```

Listing 4: Example solution dictionary

The solution dictionary should have a single key `"operations"`, whose value is another dictionary for daily operations. The daily operations dictionary has keys for dates (or time periods). For example, `"0"`, `"1"`, and `"28"` are date indices. It is not necessary to include all date indices; only dates with operations should be specified. For each date index, a list of operations is specified. There are two types of operations: `"EXIT"` and `"ENTRY"`. There is no limit on the number of daily operations. However, any `"EXIT"` operations should be performed before the `"ENTRY"` operations, as shown in the example for `"28"`'s operations.

As shown in the example, all numbers should be integers. In particular, the block's location (`"x"` and `"y"`) cannot be fractional. The evaluation system will always round the location values before conducting a feasibility check, which can be done by calling `utils.check_feasibility(prob_info, solution)`.

The baseline implements a greedy algorithm based on the EDD (earliest due date) rule. The next block to be assigned, with the earliest due date, is placed at one of the "candidate points", which are the bottom-right vertices of the AABB (axis-aligned bounding box). The resulting solution may be infeasible because the crane constraint is ignored during the first phase. The baseline algorithm simply moves the infeasible blocks to a later date, when the bays are cleared. Teams can read the baseline code and use it as a starting point.

4.2.1 Using Other Programming Languages

Using non-Python languages and libraries is allowed. You must include all required binaries and libraries in the zip and call them from `myalgorithm.py`. To maintain fairness and avoid altering the global execution environment, organizers should carefully consider installing system-wide languages and libraries. For non-Python implementations, teams are responsible for localizing everything in their submission so it does not affect other teams and for ensuring compatibility with the server environment presented in Section 3.2. Some general instructions for other non-Python languages are as follows:

- C/C++: Compile on Linux/Ubuntu 24.04 first, then submit the generated .so or executable in the zip. Server-side compilation is not performed.
- Java: The following Java runtime is available on the evaluation server

```
openjdk 17.0.18-internal 2026-01-20
OpenJDK Runtime Environment (build 17.0.18-internal+0-adhoc.conda.src)
OpenJDK 64-Bit Server VM (build 17.0.18-internal+0-adhoc.conda.src, mixed
    mode, sharing)
```

- Dotnet framework: The following Dotnet runtime is available on the evaluation server

```
Host:
  Version:      8.0.15
  Architecture: x64
  Commit:       50c4cb9fc3
  RID:          linux-x64
```

4.3 Algorithm Tester and Solution Visualization

This section introduces a simple way to test your algorithm and visualize the solutions. First, download and unzip `alg_tester.zip` from the competition website to an appropriate folder. Then, activate the `ogc2026` conda environment and run `python alg_tester_app.py`. You will see a window similar to Figure 9. Select the problem instance to solve, then specify the *directory* that contains your algorithm files, including `myalgorithm.py`. You can see the problem, bay, and block details in the "Problem" tab. The "Solution" tab will show your selected algorithm's solution to the problem. In particular, the arrangement of blocks in the bays is visualized with cross-sectional views. Note

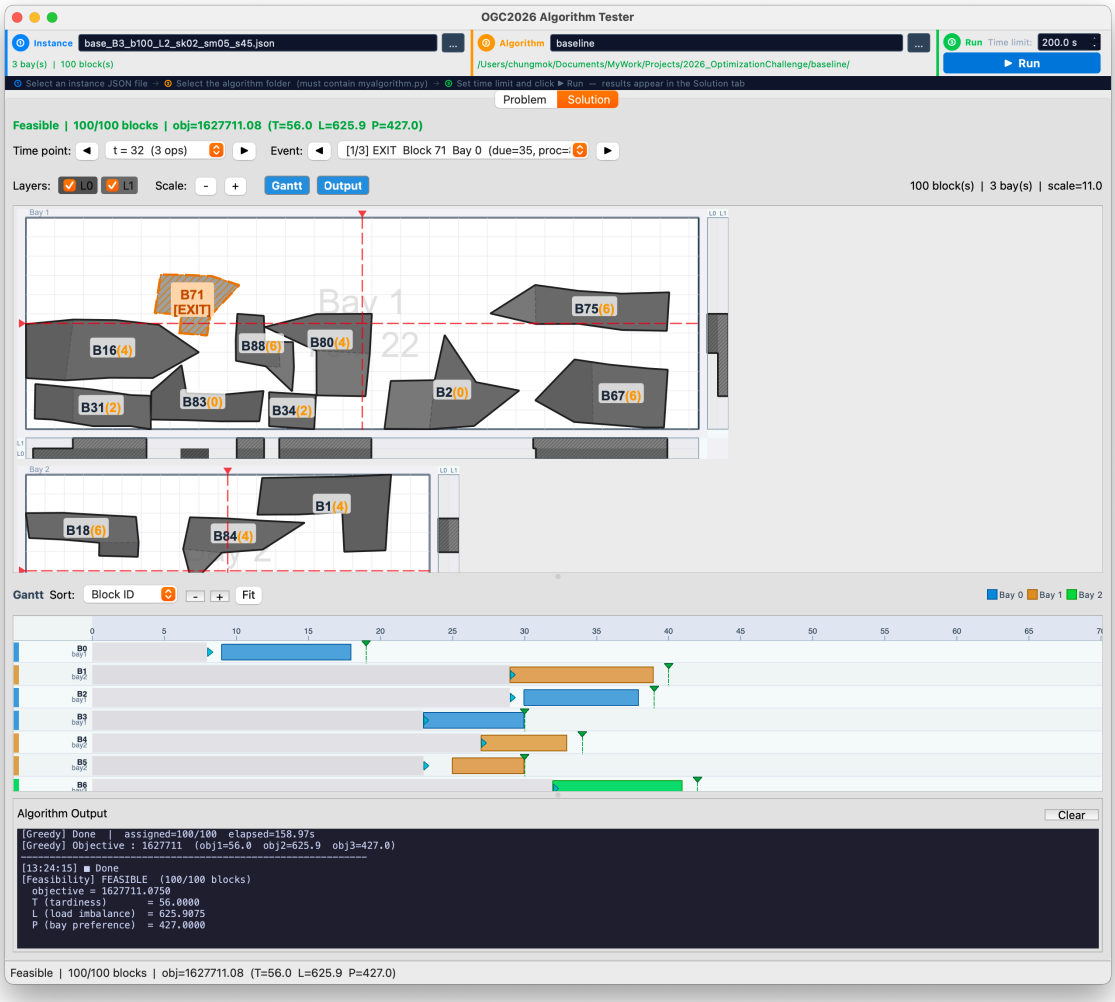

Figure 9: Algorithm Tester

that this application is provided as a reference that may be helpful. It would be appreciated if the teams could report any bugs encountered when using this application and the baseline algorithm.

5 Competition Format and Policy

The Optimization Grand Challenge 2026 is conducted in two stages: a preliminary stage and a final stage. Participants are free to employ any methodology—including mathematical optimization, machine learning, heuristics, or hybrid approaches—to solve the problem.

Leaderboard policy. Throughout each stage, the leaderboard displays standings by tier (e.g., top 10, top 20, top 30) rather than by exact rank. Precise final rankings for each stage are revealed only after all evaluations for that stage have been completed.

5.1 Preliminary Stage

1. **Automated evaluation.** Participants submit their algorithm code to the competition server, which automatically executes the code and evaluates performance. The leaderboard is updated to reflect the latest tier standings.
2. **Advancement to the final stage.** Based on the automated evaluation results and code verification conducted by the organizing committee, the top 30 to 40 teams (subject to change) are selected to advance to the final stage. Final preliminary rankings are announced upon the conclusion of this stage.

5.2 Final Stage

1. **Automated evaluation.** As in the preliminary stage, participants submit their algorithm code to the competition server for automated execution and tier-based scoring.
2. **Technical report submission.** All finalist teams are required to submit a technical report describing their algorithm in a designated format. The integrity and quality of the submitted code, together with the technical report, are reviewed separately by the organizing committee.
3. **Presentation.** Finalist teams are required to present their approach and results before the organizing committee, which is also taken into account in the final evaluation.
4. **Final selection.** Once all evaluations are complete, the organizing committee conducts a comprehensive review of the automated scores, technical reports, code evaluations, and presentations to determine the final prize-winning teams. The complete final rankings are announced at this point.

5.3 Evaluation, Disclosure, and Integrity

The specific evaluation criteria and procedures are determined and administered at the discretion of the organizing committee. Apart from the final rankings of prize-winning teams, individual evaluation results will not be disclosed.

All algorithm code and technical reports submitted by finalist teams will be publicly disclosed; accordingly, all submitted code must be eligible for release as open source. By participating, teams consent to such disclosure.

Even after the final results are announced, awards may be revoked if any misconduct—including but not limited to plagiarism, undisclosed use of restricted resources, or other violations of competition rules—is discovered or reported and substantiated through review by the organizing committee.

6 Organizing Committee

The Optimization Grand Challenge 2026 is organized and operated by

- Kyungsik Lee (Seoul National University, Chair)
- Chungmok Lee (Hankuk University of Foreign Studies)
- Jonghoon Woo (Seoul National University)
- Chankmug Kang (Soongsil University)
- Seulgi Joung (Ajou University)
- Yunwoo Jung (LG CNS)
- Gibeak Ahn (LG CNS)
- Jerimi Lee (LG CNS)